

GENETIC ALGORITHM SOLUTION OF ECONOMIC DISPATCH
WITH VALVE POINT LOADING

David C. Walters
Member
Eastman Chemical Co.
Kingsport, Tennessee 37662

Gerald B. Sheble
Senior Member
Iowa State University
Ames, Iowa 50011

Keywords: power systems, optimization, genetic algorithms, economic dispatch, valve-point loading, nonlinear programming.

ABSTRACT

A genetics-based algorithm is used to solve an economic dispatch problem for valve point discontinuities. The algorithm utilizes payoff information of candidate solutions to evaluate their optimality. Thus, the constraints of classical LaGrangian techniques on unit curves are circumvented. The formulations of an economic dispatch computer program using genetic algorithms are presented and the program's performance using two different encoding techniques is compared. The results are verified for a sample problem using a dynamic programming technique.

I. INTRODUCTION

Power systems analysis combines a highly nonlinear and computationally difficult environment with a need for optimality. Artificial intelligence, unlike strict mathematical methods, has the apparent ability to adapt to nonlinearities and discontinuities found commonly in physical systems.

The linearizations and assumptions made in the economic dispatch problem present a classic example. Classical LaGrangian techniques in economic dispatch require the unit input-output curves to be of a monotonically increasing nature. Most industrial algorithms require the incremental cost curves to be piecewise-linear. The input-output characteristics produced by generator operation can be made to approximate this requirement. But the loss of accuracy induced by these approximations is not desirable.

The genetic algorithm emulates the optimization techniques found in nature. This optimization algorithm does not require the strict continuity of classical search techniques, but allows non-linearities and discontinuities to appear in the solution space. The application of this algorithm to the economic dispatch problem uses the payoff information of an objective function to determine optimality. Therefore any type of unit characteristic cost curve may be used

with adjustments only to the objective function.

Quadratic input-output curves were used initially to develop the software and to verify that optimal solutions are obtained with this algorithm. Then linear incremental cost curves were introduced to verify the program's ability to solve the classical problem. Each curve type was also tested incorporating system transmission losses. Finally, nonlinear curves approximating valve-point characteristics were used. The results from each case were verified using separate computer programs. The valve point dispatch program uses dynamic programming while the classical models use search techniques. The results using valve-point discontinuities are presented in this paper.

II. GENETIC ALGORITHMS (GAs)

Genetic algorithms are essentially search algorithms based on the mechanics of nature (e.g. natural selection, survival of the fittest) and natural genetics. They combine solution evaluation with randomized, structured exchanges of information between solutions to obtain optimality. Genetic algorithms are considered to be robust methods because no restrictions on the solution space are made during the process. The power of this algorithm comes from its ability to exploit historical information structures from previous solution guesses in an attempt to increase performance of future solution structures [9].

GAs evaluate encodings of the original parameters instead of the actual parameters. The parameter set is reduced to a series of representative symbols from an arbitrary yet effective alphabet and solutions are evaluated based on certain symbol structures. GAs maintain a population of solution structures throughout the process, therefore they are not limited by the selection of initial single point solution guesses. In this way the entire solution space may be considered and multiple solutions detected. GAs incorporate probabilistic transition rules rather than simple deterministic rules. The programmer only has to define the objective function and the encoding techniques [9].

Solution Modeling

GA algorithms borrow the analogous biological terms for each step. GAs maintain a population of parameter set solutions and iterate on the complete population. Each iteration is called a generation. The problem parameter set, including its environment, inputs, and outputs, is represented by a fixed length string of symbols, usually from the binary alphabet {0,1}. The string, called a chromosome, represents a single solution point in the problem space. The chromosome string consists of genetic material in specific

92 SM 414-3 PWRs A paper recommended and approved by the IEEE Power System Engineering Committee of the IEEE Power Engineering Society for presentation at the IEEE/PES 1992 Summer Meeting, Seattle, WA, July 12-16, 1992. Manuscript submitted September 3, 1991; made available for printing May 13, 1992.

locations, called loci. Each location contains a symbol or series of symbols, called genes, which assume values, called alleles [10].

Evaluation of the chromosome string is accomplished by decoding the encoded symbols and calculating the objective function for the problem using the decoded parameter set. The result of the objective function calculation is used to calculate the value of the string with respect to all other chromosome strings within the population. This raw value measure is called the string's fitness value and can be calculated in any number of ways based on the goal of the optimization. For maximization problems, the fitness value could assume the value of the objective function or payoff cost itself. For minimization problems, a folding function could be used.

Representation of the problem parameter set is important. The encoding must be designed to utilize the algorithm's ability to transfer information between chromosome strings efficiently and effectively [9].

Genetic Operators

Movement in the simple genetic algorithm is accomplished using three primary operators: reproduction, crossover, and mutation. In a sample execution cycle, the reproduction operator selects a chromosome string from the previous generation based on the string's fitness and its probability of propagation to the next generation. Reproduction continues until the population of the next generation is filled. The crossover operator works in conjunction with reproduction. Crossover selects a locus position within two parent strings and swaps the gene information from that position to the end of the string. The mutation operator changes random allele values during the reproduction-crossover phase. The probabilities of crossover and mutation occurrence during an execution are selectable [10].

Execution parameter selection is important in allowing the genetic algorithm the ability to explore the entire solution space. The bulk of unbounded solution space search takes place within the first few generations of the process. The initial population is usually created using a random number generator, then subsequent generations are formed from the initial population information. Therefore it is important to provide a wide variety of genetic material to work from in the base population. Thus the population size must be chosen large enough to supply sufficient genetic structures to allow unrestricted solution space search.

III. ECONOMIC DISPATCH (ED)

The standard ED problem can be described mathematically as solving the following

$$\sum_{i=1}^n P_i = P_L + P_{TL} \quad (1)$$

where

P_i = output generation for unit i ,

n = number of generators in the system,

P_L = total current system load, and

P_{TL} = total system transmission losses.

While minimizing the objective function

$$\text{Obj} = \sum_{i=1}^n F_i(P_i) \quad (2)$$

where

OBJ = objective cost function, and

F_i = cost function for unit i .

The economic dispatch is further constrained by the following equation

$$P_i^{\min} \leq P_i \leq P_i^{\max} \quad (3)$$

where

$P_i^{\min}$ = minimum operating output of unit i ,

and

$P_i^{\max}$ = maximum operating output of unit i .

The solution is restricted to the current on-line generating units only [2].

Input-output Curves

The generator cost function is obtained from data points taken during "heat run" tests, when input and output data is measured as the unit is slowly varied through its operating region [3]. Wire drawing effects, occurring as each steam admission valve in a turbine starts to open, produce a rippling effect on the unit curve. These "valve-points" are illustrated in Figure 1 [4]. Smooth quadratic function approximations of the unit input-output characteristics provide the basis for most classical economic dispatch techniques. Thus, the valve point effects are ignored. This introduces some inaccuracy into the resulting dispatch. Methods which avoid this approximation of the actual unit curve data without sacrificing computational time would prove very valuable.

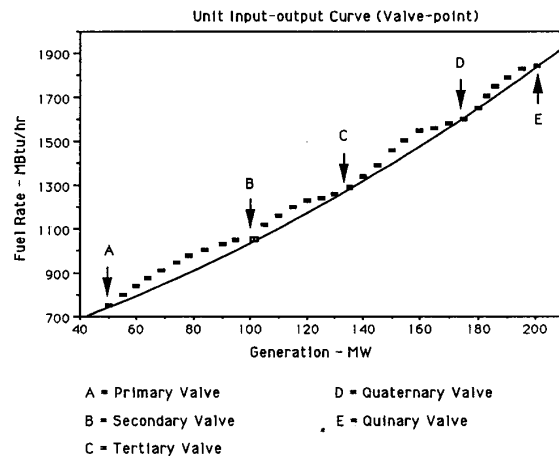


Figure 1. Example Valve-point Curve.

Solution Techniques

Classical economic dispatch techniques involve the use of advanced calculus methods relying on the LaGrangian function. The LaGrangian function adds the constraint function, multiplied by an unknown constant

variable λ , to the objective function. Many classical techniques exist to solve this well behaved formulation [5].

A classical approach to solve the ED problem with valve point modeling is Dynamic Programming (DP). Dynamic programming works with the unit input-output information directly by enumerating all possible solutions. DP consists of evaluating the problem in stages (corresponding to each unit), choosing the optimal state (corresponding to unit dispatch) for a stage by recursively examining paths from previous stages, then proceeding to the next stage until a complete optimal path is found through all stages [6].

Dynamic programming is a form of complete enumeration of the entire solution space. Solution accuracy depends on the state increment value in each stage. To achieve acceptable accuracy, each unit's dispatch must be costed for each fraction of generation in the unit's operating range. The dimensions of this problem become extremely large (curse of dimensionality). Even though multiple optimal paths may be obtained, each stage may require excessive evaluations of the recursive relationship [7].

Transmission Losses

Transmission losses are a function of the unit generations and are based on the system topology. Solving the ED equations for a specified system load requires an iterative approach since all unit generation allocations are embedded in the equation for each unit [7].

In practice, the loss penalty factors are usually obtained using on-line power flow software. This information is updated periodically (usually every few minutes) to ensure accuracy. They also can be calculated directly using the B matrix loss formula. B coefficients are used for this example.

IV. EXAMPLE APPLICATION

A computer program has been developed to perform economic dispatch of power generation using a basic genetic algorithm. The program, named SGA for Simple Genetic Algorithm, was written in the C programming language to take advantage of C's bitwise manipulations. SGA was designed to allow the user to use the genetic algorithm for any type of optimization problem through a user's interface subroutine. For the economic dispatch problem, this subroutine contains the decoding and fitness evaluation functions.

A three unit test system was obtained and used in the development of the program. To compare solution tendencies to classical economic dispatch technique results, two types of curve data and their corresponding objective functions were used: input-output curves with simple payoff comparison, and incremental cost curves with matching lambda constraint comparison. Two types of encoding schemes were examined for each curve type. Separate runs were made, first neglecting transmission losses, then including them in the form of B coefficients.

Valuable parameter information, inherent to the size and resolution of the problem space, was obtained using these preliminary runs. The resulting improvement in parameter selection was then applied to a multi-modal valve-point curve problem.

Unit Data

The example system contains a set of three generators in a simple six bus power system [5]. For values of unit generation P_i , quadratic input-output curve data was obtained using the following equation for each generator:

$$F_i(P_i) = a_i P_i^2 + b_i P_i + c_i \quad (4)$$

Incremental cost curve data was obtained by taking the derivative of the unit input-output equation (above) resulting in the following equation for each generator:

$$\frac{dF_i(P_i)}{dP_i} = d_i P_i + b_i \quad (5)$$

To model the effects of valve-points, a recurring rectified sinusoid contribution was added to the input-output equation. The result is:

$$\hat{F}_i(P_i) = F_i(P_i) + |e_i \sin(f_i(P_i - P_i))| \quad (6)$$

The constraints for these equations, as well as the unit operating ranges for the example data, are found in Table 1 below.

Table 1. Example Unit Data				
Parameter	Unit 1	Unit 2	Unit 3	
Maximum:	600 MW	400 MW	200 MW	$\bar{P}_i$
Minimum:	100 MW	100 MW	50 MW	$\underline{P}_i$
Input-output curve:				
a_i	0.001562	0.00194	0.00482	
b_i	7.92	7.85	7.97	
c_i	561	310	78	
Incremental cost curve:				
d_i	0.003124	0.00388	0.00964	
Input-output (valve-point) curve:				
e_i	300	200	150	
f_i	.0315	.042	.063	

We assume that cost per unit fuel is the same for all units, thus we may equate thermal heat content used to actual generation costs for comparison issues.

The transmission loss coefficients (B coefficients) used for the power system follow [5]:

$$[B] = \begin{bmatrix} 0.06760 & 0.00953 & -0.00507 \\ 0.00953 & 0.05210 & 0.00901 \\ -0.00507 & 0.00901 & 0.29400 \end{bmatrix},$$

$$B_0 = \begin{bmatrix} -0.07660 \\ -0.00342 \\ 0.01890 \end{bmatrix},$$

$$B_{00} = 0.040357$$

Penalty factors may be used instead of B coefficients.

Encodings

Two types of encoding schemes have been developed. Both utilize the binary alphabet {0,1}. We are interested in optimizing each unit's output, therefore each chromosome string contains an encoding of the solution for the unit generation. Illustrations of these encodings are shown in Figure 2.

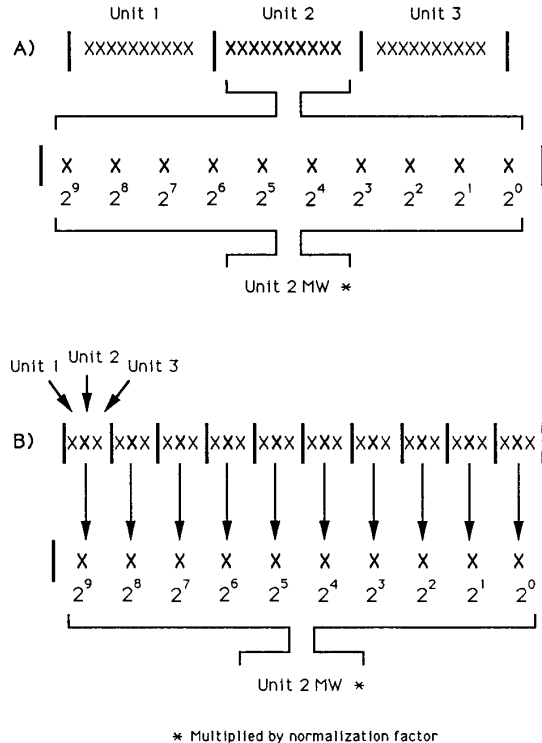


Figure 2. Encoding Illustrations

The first encoding simply stacks each unit's output value (in MW) structure in series with each other in the string. Each unit's output gene structure is assigned the same number of loci within the string. The decoded value is the actual unit output value in raw counts, based on the binary number system (powers of two) normalized over the unit's operating region. The raw counts are converted to a decimal value in the decoding process.

The second encoding scheme uses the same system for representation and decoding as the first, except that the assigned gene structures are embedded within each other throughout the string. The string is made up of a series of smaller gene structures, each containing one gene locus for each unit. Each gene structure correspondingly represents a power of two based on the binary number system.

Objective and Fitness Functions

Implementation of an objective function and constraints in a genetic algorithm are realized within the fitness function. The fitness function acts as a pseudo objective function, since it is a raw measure of the solution value. Inclusion of the constraints in the fitness function requires the introduction of tolerances in satisfying the constraint equations. However, if the fitness function and encoding are well designed, accuracy will not suffer.

The economic dispatch problem can be solved by the genetic algorithm using either the unit input-output or incremental cost curves. The input-output curve solution uses the standard objective function and a penalty term for the conservation of energy constraint. Unit limits are automatically satisfied by the normalization process in the chosen encoding described previously, which only allows solutions in the operating range of the units.

The constraint equation can be rewritten as:

$$lerr = \left| \sum_{i=1}^n P_i - P_L - P_{TL} \right| \quad (7)$$

which when reduced to zero satisfies the original constraint.

Since a chromosome string's fitness will be compared to all other string fitnesses within the same population, an absolute measure of optimality is not required. Likewise, the constraint equation error can be calculated as a comparison within the same population. The result is a fitness function based on a percentage rating. The following forms were used:

$$\%obj = \frac{stringobj - minobj}{maxobj - minobj} \quad (8)$$

and

$$\%lerr = \frac{stringerr - minerr}{maxerr - minerr} \quad (9)$$

where %obj is a percentage value of the objective function, stringobj is string's fitness (objective function) value, minobj is the minimum objective function value within the population, maxobj is maximum objective function value within the population, %lerr is percentage value of the string's constraint equation error, stringlerr is string's error in meeting the constraint equation, minerr is minimum constraint error within the population, and maxerr is maximum constraint error within the population.

The fitness function objective then becomes:

$$Obj = MIN \left[\%obj - \%lerr \right] \quad (10)$$

Since the fitness function is a raw value measure of the string's fitness, we desire to give the best solutions the highest fitness values while minimizing the function above. Simply change the minimization into maximization by folding the fitness about the value one. If algorithm control or emphasis to certain problem objectives or constraints is desired, scaling factors are

added. The final form of the fitness function becomes:

$$\text{Fit} = sf_1 \left[(1 - \%obj) (sp_1) \right] + sf_2 \left[(1 - \%lerr) (sp_2) \right] \quad (11)$$

where sf is a scaling factor for emphasis within the function itself and sp is a scaling power factor for emphasis over the entire population.

The incremental cost curve representation replaces the objective function with the following constraint:

$$\frac{dF_1(P_1)}{dP_1} = \dots = \frac{dF_n(P_n)}{dP_n} = \lambda \quad (12)$$

This added constraint can be handled in the same fashion as the conservation of energy constraint. An error term is then introduced and a measure of this error is calculated. The constraint is met if each term in the equality is equal to the average lambda value for the string. Therefore, the error term is:

$$\lambda_{err} = \sum_{i=1}^n \left| \lambda_{avg} - \lambda_i \right| \quad (13)$$

where λ_{err} is the error term, λ_i is the incremental cost for unit i , and λ_{avg} is the average incremental cost value for the string. $\%lerr$ is then calculated similarly to $\%obj$ in equation (8).

The fitness objective becomes

$$\text{Obj} = \text{MIN} \left[\%lerr - \%lerr \right] \quad (14)$$

and the fitness function is

$$\text{Fit} = sf_1 \left[(1 - \%lerr) (sp_1) \right] + sf_2 \left[(1 - \%lerr) (sp_2) \right] \quad (15)$$

V. SOLUTIONS AND PERFORMANCE

For each data set case, four separate runs were made to demonstrate the ability of the program using different initial random seed numbers. The cases considered were:

- A) Input-output curves neglecting losses.
- B) Input-output curves considering losses.
- C) Incremental heat rate curves neglecting losses.
- D) Incremental heat rate curves considering losses.
- E) Valve-point curves neglecting losses.

For each case, both encodings were tested. After evaluation and testing of different parameter combinations, the SGA program parameters for all cases were held constant and were chosen to be:

- 100 - Generations.
- 100 - Strings per generation.
- 11 - Bit representation per unit.
- .95 - Percentage occurrence of crossover.
- .01 - Percentage occurrence of mutation.

The fitness scaling parameters for cases A through D were chosen to be:

- 25 - Load constraint scaling factor (sf_1).
- 15 - Cost scaling factor (sf_2).
- 3 - Scaling power factor (sp_1 & sp_2).

and for case E:

- 4.5 - Load constraint scaling factor (sf_1).
- 2.0 - Cost scaling factor (sf_2).
- 3 - Scaling power factor (sp_1 & sp_2).

For each case, a table was created showing the best string solution of the final generation for all four runs. Also shown was the optimal solution obtained from a verification program using dynamic programming techniques. Figures showing the best solution and the average solution of the population for each generation, as well as the final generation solutions with respect to the optimal solutions were provided [1]. Only the valve-point curves are shown within this paper.

Valve-point Curves

The genetic algorithm solution results of each encoding for valve-point curves neglecting transmission losses, along with the optimal solutions found by our verification program, are shown in Tables 2 and 3 below. The tables demonstrate the ability of the genetic algorithm technique using encoding A to reliably converge to the optimal solutions of cost and system load demand. The selection of scaling parameters is the suspected cause of encoding B's poorer performance. Preliminary experimentation with the scaling parameters has provided some insight for this assumption.

Table 2. Valve-point - Encoding A

RUN	UNIT 1	UNIT 2	UNIT 3	LOAD	COST
1	498.4	250.4	100.5	849.3	8249.0
2	299.1	399.0	150.8	848.9	8239.2
3	398.7	399.6	50.1	848.4	8240.3
4	392.1	307.4	149.7	850.2	8425.7
*	300.0	400.0	150.0	850.0	8237.6
	(MW)	(MW)	(MW)	(MW)	(\$)

Table 3. Valve-point - Encoding B

RUN	UNIT 1	UNIT 2	UNIT 3	TOTAL	COST
1	598.0	176.5	75.7	850.2	8495.2
2	499.1	250.4	99.9	849.4	8232.2
3	498.6	250.2	87.2	836.0	8221.2
4	498.9	204.6	147.5	851.0	8468.9
*	300.0	400.0	150.0	850.0	8237.6
	(MW)	(MW)	(MW)	(MW)	(\$)

Figures 3 through 6 show the converging performance of the genetic algorithm program during each of the four runs. The best string solution of the population was selected based on the fitness function value for each generation of the run. The constraint equation solution and the cost function solution of the best solution string is plotted for both encodings. The figures also show the optimal solution for the example plotted as a straight line.

Encoding A demonstrates a strict convergence in three of the four runs while

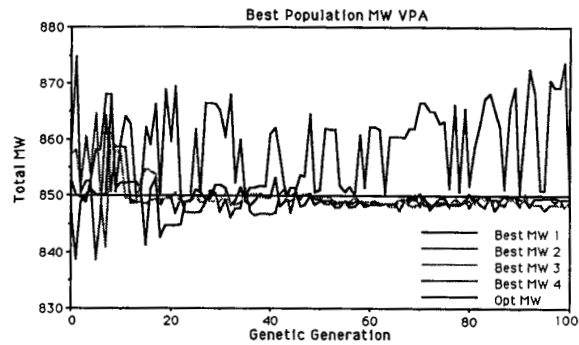


Figure 3. Best Generation - Encoding A

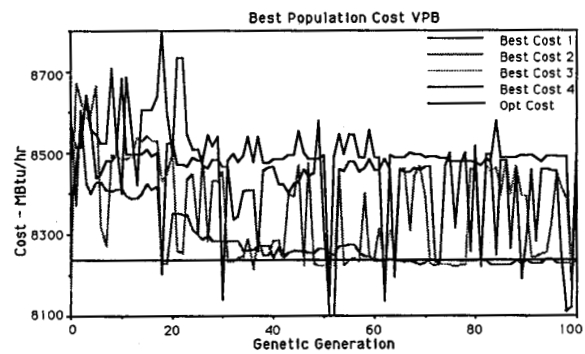


Figure 6. Best Cost - Encoding B

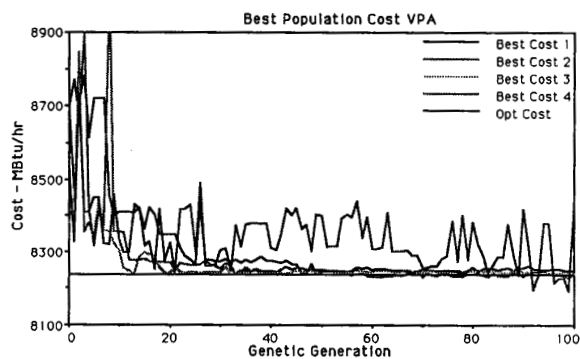


Figure 4. Best Cost - Encoding A

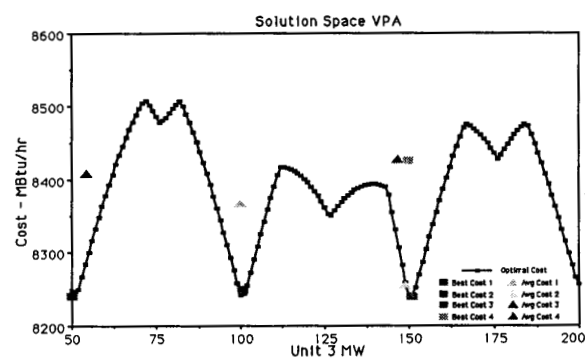


Figure 7. Solution Space - Encoding A

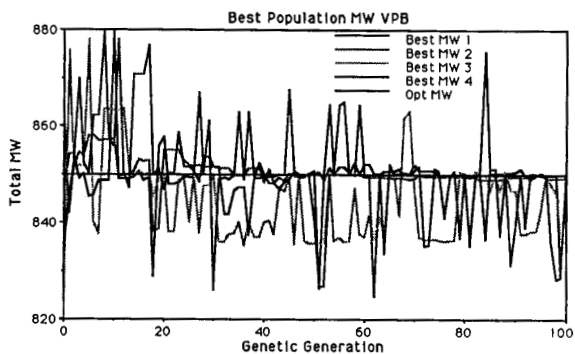


Figure 5. Best Generation - Encoding B

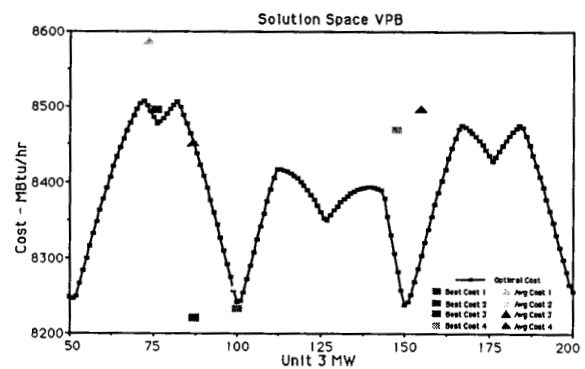


Figure 8. Solution Space - Encoding B

encoding B converges to the optimal in only one of the four runs. Also, it is worth noting that the algorithm tends to settle to the optimal solution within 50 generations when the best string results are indeed optimal for the problem.

Figures 7 and 8 illustrate the final results of each run on the solution space for the problem. The graphs show the best cost dispatch for all three units versus a set MW output of unit 3. From this graph, we see that valve-points cause the solution space to be multi-modal. The absolute optimal solution occurs at a setting of 150 MW for unit 3. The small rectangles show the final best costs of each run. Also shown as small triangles is the average of all string solutions in the final generation. These graphs demonstrate the level of difficulty of the problem and the ability of the algorithm to locate the optimal solutions using encoding A. The level of convergence to a solution can also be deduced from these graphs by the proximity of the average cost to the best cost of a run.

The CPU time used by the GA program for this example averaged 16 seconds compared to 122 seconds used by the dynamic programming verification program (timed on an IBM Model 70). Comparison of run-times for this example are misleading. As more units are added or incremental values of MW is increased, dynamic programming time will increase geometrically while genetic algorithm time will increase linearly. Note that the genetic algorithm was allowed a larger population size and more generation per run than required in the above figures. It should also be noted that additional constraints may be added to the GA program very easily. Additional constraints may not be added to the DP program without major software changes.

VI. CONCLUSIONS

Either encoding has been shown to be a proper encoding based on the results of this research. The effort required to initially find the algorithm parameters requires several runs to determine the settling time and will vary slightly as models and constraints are changed or added. Once the algorithm is tuned for the models and constraints, no additional maintenance effort is needed.

The results show that the genetic algorithm is a powerful optimization tool that can be used to solve economic dispatch. The advantage of the genetic algorithm lies in its ability to handle any type of unit characteristic data, whether smooth or not. A more exact model for the unit characteristics can be used with this method. The genetic algorithm approach yields solutions which are very near optimal. Selection of scaling parameters in the fitness function evaluation equation is a crucial initial tuning process. Scaling parameter selection may present a major obstacle in solving more complex problems for which there are no known solutions for comparison.

The simple encodings used in this thesis proved to be very reliable, but additional

insight in this area could significantly reduce computer requirements. The key factor in the genetic algorithm's tendency to find optimal solutions lies in its ability to distinguish the fitness of optimal solutions, thus the application of other penalty functions could yield significant improvement. A method for testing convergence must also be accepted.

REFERENCES

1. David C. Walters, An Application of Genetic Algorithms to the Economic Dispatch of Power Generation, MS Thesis, Auburn University, May, 1991.
2. Gross, C.A., Power Systems Analysis. John Wiley & Sons, New York, NY, 1986.
3. Noyola, A.H., Grady, W.M., Viviani G.L., "An Optimized Procedure for Determining Incremental Heat Rate Characteristics," IEEE Transactions on Power Systems, Vol. 5, May 1990, pp 376-383.
4. IEEE Committee Report, "Present Practices in the Economic Operation of Power Systems," IEEE Transactions on Power Apparatus and Systems, Vol. PAS-90, July/August 1971, pp. 1768-1775.
5. Wood, A.J. and Wollenberg, B.F., Power Generation, Operation, and Control, John Wiley & Sons, New York, NY, 1984.
6. Shoults, R.R., et al, "A Dynamic Programming Based Method for Developing Dispatch Curves when Incremental Heat Rate Curves are Non-Monotonically Increasing," IEEE Transactions on Power Systems, Vol. PWR-1, February 1986, pp. 10-16.
7. Ross, D.W., Kim, S., "Dynamic Economic Dispatch of Generation," IEEE Transactions on Power Apparatus and Systems, Vol. PAS-99, Nov/Dec 1980, pp. 2060-2068.
8. Ramanathan, R., "Fast Economic Dispatch Based on the Penalty Factors From Newton's Method," IEEE Transactions on Power Apparatus and Systems, Vol. PAS-104, July 1985, pp. 1624-1629.
9. Goldberg, D.E., Genetic Algorithms in Search, Optimization & Machine Learning. Addison-Wesley, Reading, MA, 1989.
10. Austin, S., "An Introduction to Genetic Algorithms," AI Expert, Vol. 5, March 1990, pp. 48-53.

David C. Walters (Student Member) is an Electrical Engineer for Tennessee Eastman Company, Kingsport, Tennessee. Mr. Walters received his B. and M.S. degrees in Electrical Engineering from Auburn University. His industrial experience includes one year at Scott Paper Company in Mobile, Alabama, as a cooperative student and two years as a Power Systems Analyst with the Controls and Composition Division of Harris Corporation, Melbourne, Florida. His current assignment is in industrial power applications.

Gerald B. Sheble (M 71, SM 85) is an Associate Professor of Electrical Engineering, Iowa State University, Ames, Iowa. Dr. Sheble received his B.S. and M.S. degrees in Electrical Engineering from Purdue University and his Ph.D. in Electrical Engineering from Virginia Tech. His research interests include distribution and power system optimization, scheduling and control.

Discussion

M. E. El-Hawary (Technical University of Nova Scotia, Halifax, N.S., Canada): This is a welcome addition to the growing literature on the application of "nature emulating" techniques to electric power system problems. The paper treats the economic dispatch problem in an innovative manner to treat the issue of valve point loadings effectively. As an application of a technique not familiar to the power systems audience, a number of questions arise, and the authors' clarifications are appreciated.

- In explaining the choices available for a fitness function, the authors state that a folding function could be used in case of minimization. Could the authors elaborate on this point further?
- Economic dispatch can be thought of as a process of maximizing the negative of the operating costs, and hence according to the available options for selecting a fitness function, one could simply use this objective directly rather than that suggested in (10). Did the authors experiment with this option?
- The forms suggested in (8) through (10) all depend on the characteristics of the population in each generation, and it would appear that the fitness function value is sensitive to the size of initial population. Are there any guidelines as to the minimum acceptable population size?

The authors should be commended for introducing this technique to the power systems literature.

Manuscript received August 3, 1992.

D.C. Walters, G.B. Sheble: We would like to thank Dr. El-Hawary for his gracious comments regarding the paper and respond to his questions in the following paragraphs.

In order to evaluate each separate percentage value within the objective function as a minimization, it was necessary to "fold" the fitness function about the value one. This simply means that the percentage was subtracted from 1.0 yielding a fitness function scale from 1.0 to 0.0. Consequently, the best fitness functions within a population received values near 1.0 while weaker functions received values near 0.0.

The original intent in developing the percentage fitness functions used in this paper was to force the cost function and the generation/load demand criteria scales to be the same. In early

experiments, we found using the raw numbers of cost in dollars and generation load error in MW or KW caused the genetic algorithm to favor one of the objectives over the other rather than weight both equally. This caused premature convergence to false peaks. The use of percentages was an attempt to force the algorithm to weight all objectives equally within the fitness function. Dr. El-Hawary's suggestion of maximizing the negative of the operating costs seems to be a valid option. But in the eventual use of a percentage evaluation of the fitness function it would not give any advantage over minimization.

Dr. El-Hawary is correct in his observation of the importance of choosing the population size correctly. In Dr. Goldberg's book on genetic algorithms [9], theory on population size is discussed but no proven formula is offered. To determine the best parameter values of population size, genetic operator frequencies, and scaling factors for this example, the optimal solution was introduced as a complete string into the initial population while these values were varied. It became apparent that by restricting the population size, the genetic operators would sometimes lose the optimal genetic material by placing it with suboptimal material and eliminating it in the next generation. It was also evident that it is important to have enough genetic material and combinations of material in the initial population to examine the entire solution space in detail especially for multimodal problems such as the one presented here. The application of statistical analysis to determine the number of combinations possible for a given encoding may give some insight into the optimal population size.

Again we would like to thank Dr. El-Hawary for his comments and questions. We hope these responses will shed additional light on the paper.

Manuscript received September 28, 1992.